

Relazione 3° Homework

Sistemi distribuiti

Gruppo:

- *Cristiano Pistorio 1000067656*
- *Giuseppe Romano 1000056390*

Sommario

Abstract.....	2
Diagramma architetturale	3
Kubernetes	4
White box monitoring.....	5
Alertnotificationsystem	7
Alertsystem	9
Datacollector	10

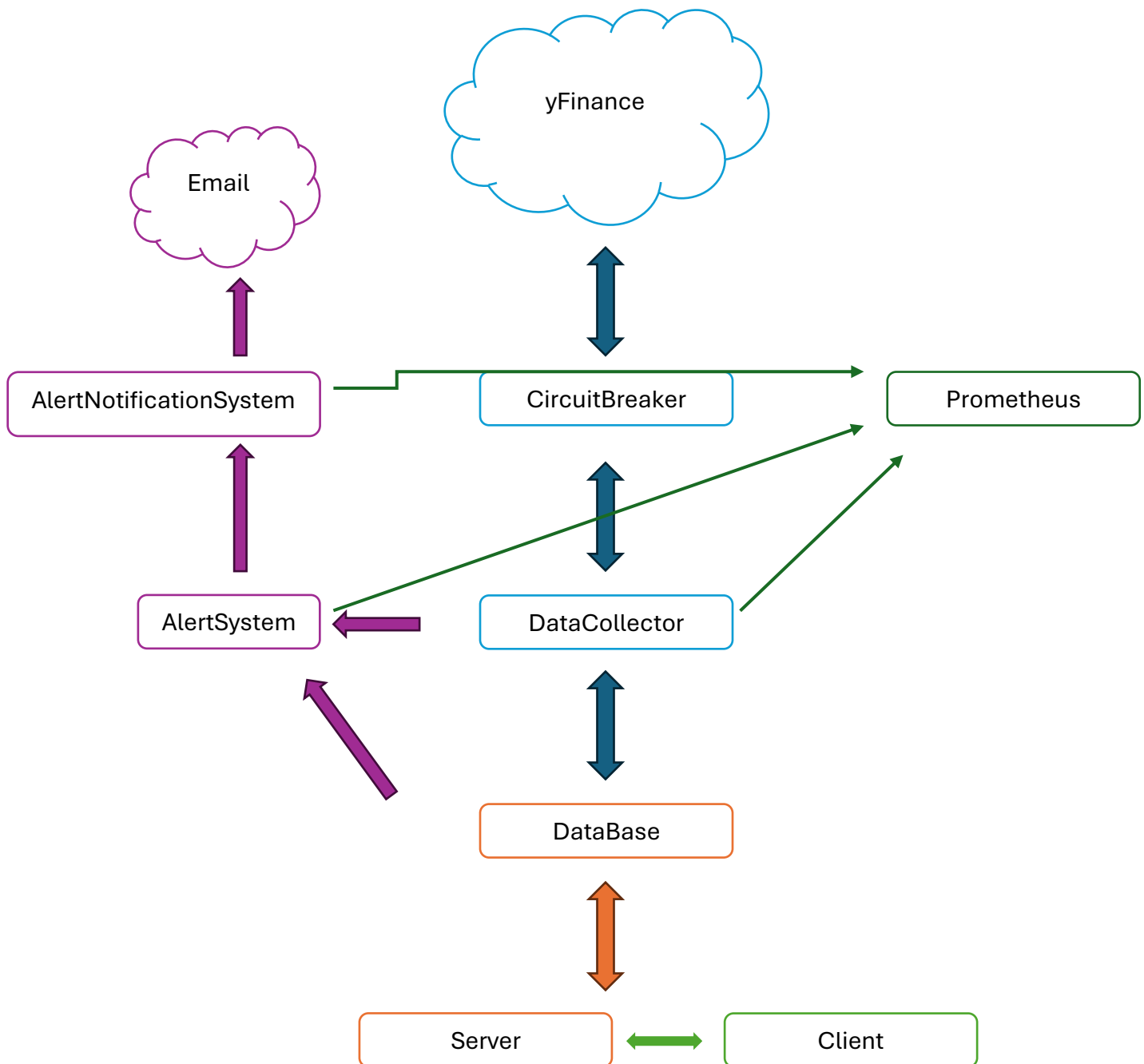
Abstract

Questo documento descrive l'evoluzione del progetto, con un focus sul porting della soluzione precedente su Kubernetes e l'integrazione di Prometheus per il monitoraggio avanzato. Il passaggio da Docker Desktop a Kubernetes rappresenta un importante miglioramento, permettendo una gestione più scalabile e robusta delle applicazioni containerizzate. È stato utilizzato kind per creare un cluster locale, caricando le immagini Docker e configurando il cluster tramite file YAML per deployment e servizi.

Un'altra novità è l'aggiunta di Prometheus, implementato per monitorare metriche chiave dei microservizi più critici: datacollector, alertsystem e alertnotificationsystem. Ogni servizio è stato arricchito con metriche dettagliate, come tempi di risposta, contatori di richieste, errori e messaggi processati. Grazie a PromQL, sono state realizzate query per analizzare i dati e generare grafici utili per comprendere meglio il comportamento del sistema.

Questo aggiornamento non solo migliora la gestione e il monitoraggio, ma pone solide basi per scalabilità e affidabilità, consolidando i progressi dei lavori precedenti con strumenti e configurazioni ottimizzate.

Le modifiche attuate rispetto all'homework 3 sono evidenziate in **verde scuro**. Si aggiunge solamente Prometheus.



Kubernetes

La parte principale del lavoro è stata fare il porting dell'HW2 in Kubernetes.

Kubernetes è una piattaforma open-source che automatizza il deployment, la scalabilità e la gestione di applicazioni containerizzate, ideale per gestire cluster di container su larga scala. Docker Desktop, che utilizzavamo precedentemente, è uno strumento per sviluppatori che consente di creare, testare e distribuire applicazioni containerizzate su un singolo computer, facilitando lo sviluppo locale.

La principale differenza tra i due è lo scopo: Kubernetes è progettato per la gestione di cluster su larga scala, mentre Docker Desktop è orientato allo sviluppo e al test su un singolo computer. Kubernetes offre funzionalità avanzate di orchestrazione come il bilanciamento del carico e il ripristino automatico, mentre Docker Desktop fornisce un'interfaccia semplice per la gestione dei container con funzionalità di orchestrazione limitate.

Per sfruttare Kubernetes abbiamo deciso di utilizzare kind: Kind (Kubernetes IN Docker) è uno strumento che consente di eseguire cluster Kubernetes localmente utilizzando container Docker come "nodi". È stato progettato principalmente per testare Kubernetes stesso.

Quindi è stato necessario prima fare il build di tutte le immagini, sempre attraverso docker.

Dopo aver fatto ciò è stato necessario fare il load sul cluster kind di tutte le immagini.

La configurazione del cluster è definita nel file [kind-config.yaml](#).

Abbiamo realizzato i file deployment.yaml e service.yaml per ogni servizio.

In Kubernetes, un Deployment è una risorsa che gestisce il deployment e la scalabilità di un set di pod. Consente di dichiarare lo stato desiderato per le applicazioni e Kubernetes si occupa di mantenere questo stato, gestendo il ciclo di vita dei pod, eseguendo aggiornamenti e rollback.

Un Pod è l'unità base di esecuzione in Kubernetes. Rappresenta un'istanza di un'applicazione o un gruppo di container che condividono lo stesso spazio di rete e di storage. I container all'interno di un pod possono comunicare tra loro facilmente e sono gestiti come un'unica unità. I pod sono effimeri, il che significa che possono essere creati e distrutti in base alle esigenze del cluster.

Un Service, invece, è una risorsa che definisce un endpoint di rete stabile per un set di pod. I Service permettono di esporre le applicazioni in esecuzione nei pod, bilanciando il carico tra di essi e garantendo che le applicazioni siano accessibili sia all'interno del cluster che dall'esterno, a seconda del tipo di Service configurato, noi abbiamo usato *NodePort* per rendere il servizio visibile dall'esterno

Per applicare le configurazioni dei file si è fatto uso dell'apposito comando fornito da kubectl.

Implementazione CQRS in K8s

Il pattern CQRS (Command Query Responsibility Segregation) è stato implementato per migliorare la scalabilità e la gestione delle risorse del nostro sistema. Questo pattern separa le operazioni di comando (che modificano lo stato) dalle operazioni di query (che leggono lo stato), permettendo di scalare indipendentemente i servizi di comando e query. Per implementarlo in K8s abbiamo creato due microservizi separati:

- **user-command-service:** Gestisce le operazioni di comando
- **user-query-service:** Gestisce le operazioni di query

questi sono implementati attraverso due container che sono “deployati” nello stesso pod che espone un unico service. In questo modo abbiamo una totale separazione dei compiti, mantenendo un’organizzazione dei servizi semplice ed intuitiva.

White box monitoring

il **white box monitoring** si riferisce al monitoraggio delle metriche interne di un'applicazione, come il numero di richieste, la latenza, l'utilizzo della CPU, ecc. Questo tipo di monitoraggio fornisce una visibilità dettagliata sul funzionamento interno dei servizi, permettendo di identificare e risolvere problemi in modo più efficace. Ad interessarci sono due tipologie di metriche: Gauge e Counter.

- **Counter:** Un counter è una metrica che rappresenta un valore che può solo aumentare (o essere resettato a zero). È utile per contare eventi come il numero di richieste HTTP ricevute o il numero di errori.
- **Gauge:** Un gauge è una metrica che può aumentare e diminuire. È utile per misurare valori che possono variare nel tempo, come l'utilizzo della memoria o la temperatura o il tempo di risposta.

Al fine di ottenere questo monitoraggio delle metriche si è aggiunto il servizio Prometheus.

Prometheus è un sistema di monitoraggio e allarme open-source progettato per raccogliere e analizzare metriche da varie fonti. È particolarmente popolare nel contesto dei microservizi e delle architetture cloud-native grazie alla sua capacità di scalare e gestire grandi volumi di dati metrici.

Noi in particolare abbiamo deciso di aggiungere metriche ai nostri microservizi più “delicati”: **datacollector**, **alersystem** e **alertnotificationsystem**. Per ognuno di essi sono state monitorate delle metriche.

Per il microservizio **datacollector** sono state monitorare due metriche di tipo counter e una metrica di tipo gauge:

- response_time_gauge
- request_counter
- error_counter

Per il microservizio **alertsystem** sono state monitorate due metriche counter e una gauge:

- request_counter
- error_counter
- processing_time_gauge

Per **alertnotificationsystema** abbiamo monitorato tre metriche counter e una metrica gauge:

- email_counter
- error_counter
- message_counter
- processing_time_gauge

Al fine della relazione riportiamo tutti i grafici delle relative metriche insieme alla query usate per averle restituite, facendo uso di **PromQL** (Prometheus Query Language), il linguaggio per interrogare e analizzare le metriche raccolte.

PromQL è specificamente progettato per lavorare con le serie temporali di Prometheus.

Permette di selezionare e aggregare le metriche, eseguire calcoli e trasformazioni sui dati, e generare visualizzazioni utili per il monitoraggio e l'analisi.

I componenti di una Query PromQL sono:

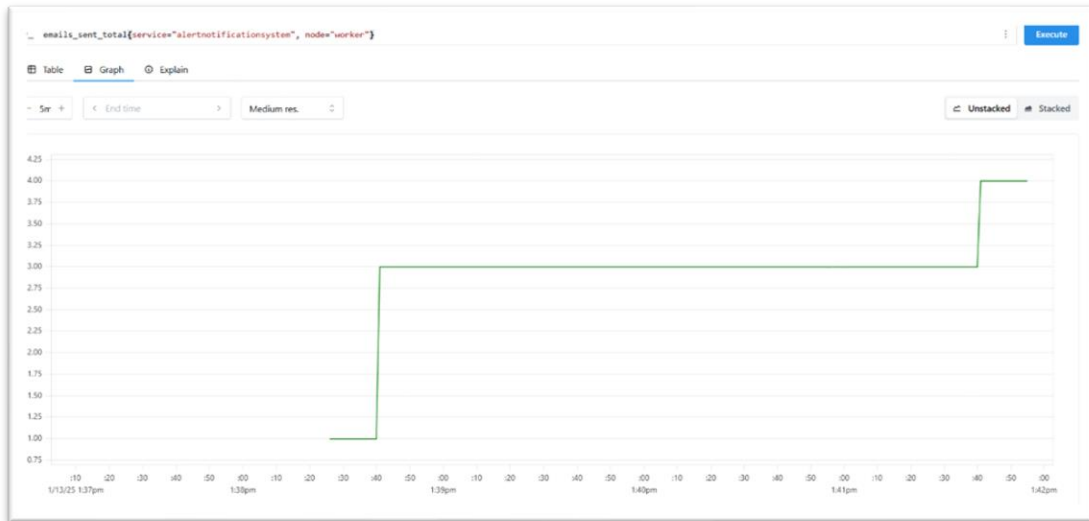
- **Selettori di Serie Temporali:** Specificano quali serie temporali devono essere selezionate. Ad esempio, `http_requests_total` seleziona tutte le serie temporali con il nome `http_requests_total`.
- **Operatori:** Permettono di eseguire operazioni matematiche e logiche sulle serie temporali. Ad esempio, `rate(http_requests_total[5m])` calcola il tasso di incremento delle richieste HTTP negli ultimi 5 minuti.
- **Modificatori:** Permettono di modificare il comportamento delle query, come offset per spostare la finestra temporale di una serie temporale.

Alertnotificationsystem

A seguire sono presenti le metriche analizzate nell'alertnotificationsystem, le query necessarie all'estrazione di esse e i grafici relativi

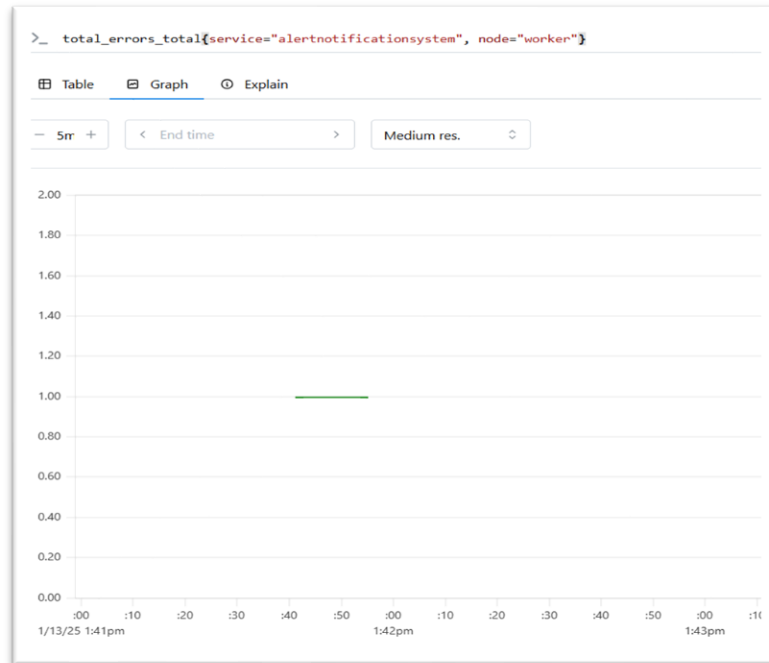
- Il numero di e-mail inviate

emails_sent_total{service="alertnotificationsystem", node="worker"}



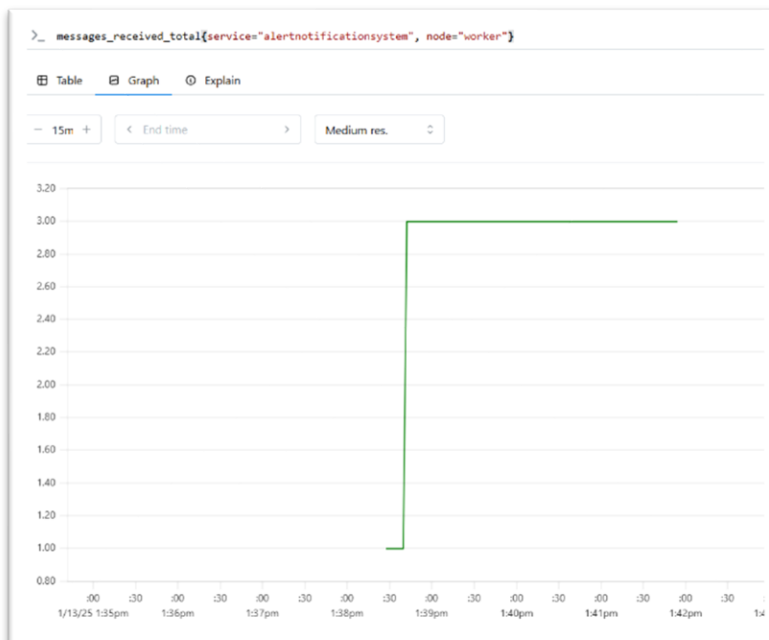
- Il numero di errori nell'alertnotificationsystem

total_errors_total{service="alertnotificationsystem", node="worker"}



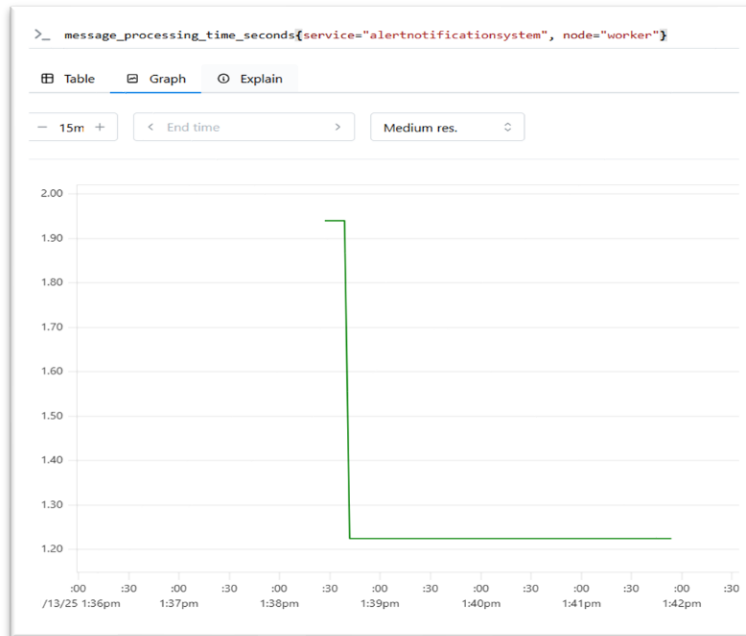
- Il numero totale di messaggi ricevuti

`messages_received_total{service="alertnotificationsystem", node="worker"}`



- Tempo di elaborazione dei messaggi

`message_processing_time_seconds{service="alertnotificationsystem", node="worker"}`

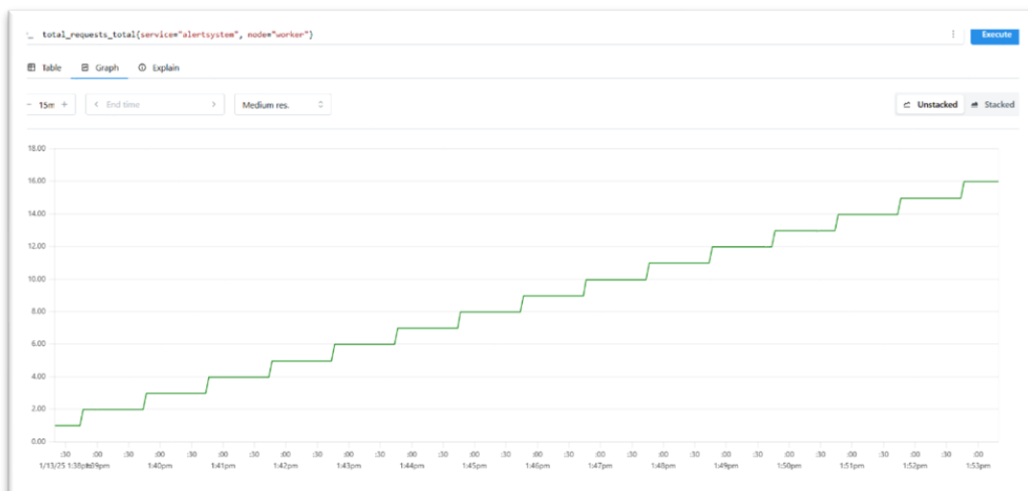


Alertsystem

A seguire sono presenti le metriche analizzate nell'alertsystem, le query necessarie all'estrazione di esse e i grafici relativi

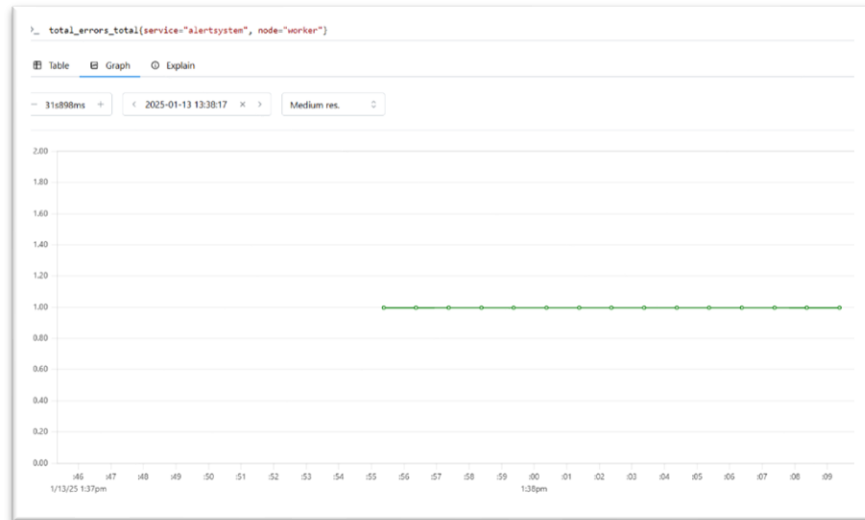
- Numero totale di richieste

total_requests_total{service="alertsystem", node="worker"}



- Numero totale di errori

total_errors_total{service="alertsystem", node="worker"}



- Tempo di processingo di un messaggio

message_processing_time_seconds{service="alertsystem", node="worker"}

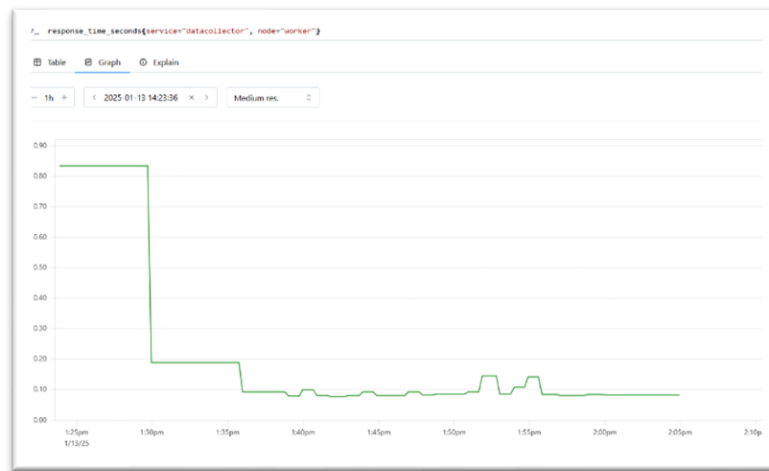


Datacollector

A seguire sono presenti le metriche analizzate nell'alertsystem, le query necessarie all'estrazione di esse e i grafici relativi

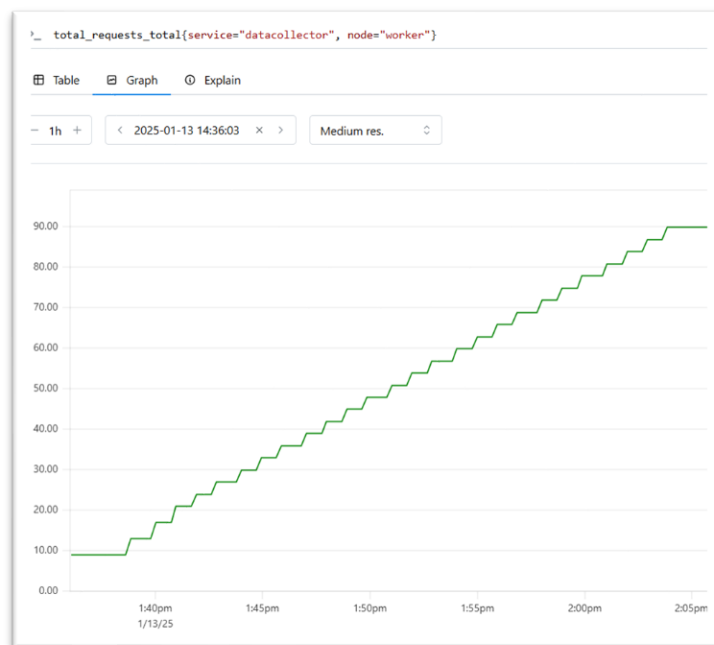
- Tempo di risposta

response_time_seconds{service="datacollector", node="worker"}



- Numero totale di richieste nel tempo

total_requests_total{service="datacollector", node="worker"}



- Numero totale di errori

total_errors_total{service="datacollector", node="worker"}

